GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 12 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Data structure and object design

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Data structure and object design | 6 questions | tap a title to jump to it

M	1. Build Prefix Lookup with a Trie	PracHub	2026-07-03
M	2. Implement a Reusable Locker Allocation System	PracHub	2026-08-14
M	3. Design a restaurant waitlist system	PracHub	2026-03-01
M	4. Design a logger that returns top-K messages	PracHub	2026-01-06
M	5. Design set with O(1) random access	PracHub	2026-01-01
M	6. Determine Reporting Relationships	PracHub	2026-02-22

STUDY PATH: question 63 of 167 (Core 90)

Priority score 3.94 | Data structure and object design | asked 109 days ago

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 1 of 6

Build Prefix Lookup with a Trie

POSTED 2026-07-03 | 109 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **ALL TIME 36** | **30 DAYS 0** | **PRIORITY 105/228**

ABOUT THIS QUESTION

Topic: Core 90 - Data structure and object design

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree) (inferred from overview)

Posted on PracHub: 2026-07-03 (updated 2026-07-04)

Original: https://prachub.com/coding-questions/build-prefix-lookup-with-a-trie

QUESTION

Overview: Practice a Google coding interview problem focused on build prefix lookup with a trie. The prompt emphasizes edge cases, clean implementation, and verifiable test behavior without revealing the solution.

APPROACH

Trie of dict words; walk the prefix, then DFS the subtree to collect completions (optionally store counts at nodes).

HOW TO THINK ABOUT IT

Prefix questions repeat work when each query scans the whole word list. A trie stores each prefix once: walk down by the query, then everything below that node is a completion. Keep counters or a word flag in the nodes to answer follow-ups such as 'how many words start with this' or 'top K completions' without another walk.

Basis: LeetCode 208 (Implement Trie) and 1268 (Search Suggestions System).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(words, prefix):
    found = []
    for word in words:
        if word.startswith(prefix):
            found.append(word)
    return found
```

TIME	SPACE
O(W * L) per query	O(1) extra

BRUTE FORCE

WHY IT IS SLOW

Every query touches every word, O(W * L). A trie walks only the prefix and then the subtree below it.

OPTIMAL SOLUTION · Trie with a DFS below the prefix node

TIME	SPACE
O(L) to walk the prefix + O(size of the answer)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many words start with this prefix?

Store a counter in each node that is incremented when a word passes through it; the answer is the counter at the end of the prefix walk, O(L) with no DFS.

FOLLOW-UP 2 Return only the top 3 completions (by frequency or alphabetically).

Keep at each node a short list of its best K words (updated on insert), or bound the DFS: visit children in alphabetical order and stop after K words. That is the LeetCode 1268 style answer.

FOLLOW-UP 3 Memory is large (many words with long common prefixes).

A compressed (radix) trie stores whole edge labels instead of single characters; a sorted array with binary search over prefixes uses the least memory for a static dictionary. **Cost:** a radix trie has O(number of words) nodes instead of O(total characters); a sorted array uses O(total characters) with O(L log W) lookups.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Build Prefix Lookup with a Trie
#
# Practice a Google coding interview problem focused on
# build prefix lookup with a trie. The prompt emphasizes
# edge cases, clean implementation, and verifiable test
# behavior without revealing the solution.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many words start with this prefix?
# 2. Return only the top 3 completions (by frequency or
# alphabetically).
# 3. Memory is large (many words with long common
# prefixes).
#
# INTERVIEW NOTES
```

```
# Prefix questions repeat work when each query scans the
# whole word list. A trie stores each prefix once: walk
# down by the query, then everything below that node is a
# completion. Keep counters or a word flag in the nodes to
# answer follow-ups such as 'how many words start with
# this' or 'top K completions' without another walk.
#
# BRUTE FORCE (say it first): Scan every word for the
# prefix
# For each query test word.startswith(prefix) on every
# dictionary word. Correct, but each query pays for the
# whole dictionary. Time O(W * L) per query. Space O(1)
# extra.


# BRUTE FORCE (say it first): Scan every word for the
# prefix | time O(W * L) per query
def completions_brute(words, prefix):
    return sorted(
        w for w in set(words) if w.startswith(prefix)
    )


# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # does a word end here?
        self.is_word = False
        # how many words pass through this node
```

```
        self.count = 0


# BASE QUESTION (optimal): Trie with a DFS below the
# prefix node
# Idea: Insert every word into nested dicts, marking word
# ends. To answer a prefix, walk down one character at a
# time (no match means no completions), then DFS the
# subtree collecting words.
# FOLLOW-UP 1: How many words start with this prefix?
# Answer: Store a counter in each node that is incremented
# when a word passes through it; the answer is the counter
# at the end of the prefix walk, O(L) with no DFS.
class PrefixTrie:
    def __init__(self, words=()):
        self.root = Node()
        for w in words:
            self.insert(w)

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
            node.count += 1
        # a repeated word is only counted as one word
        if not node.is_word:
            node.is_word = True
        else:
            self.undo_count(word)

    def undo_count(self, word):
        node = self.root
        for ch in word:
            node = node.children[ch]
            node.count -= 1

    def walk(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
```

```
        return node

    # FOLLOW-UP 1: how many words start with the prefix
```

```
    def count_prefix(self, prefix):
        # how many words start with the prefix: O(len)
        node = self.walk(prefix)
        if prefix == "":
            return self.root_count()
        return node.count if node else 0

    def root_count(self):
        kids = self.root.children.values()
        return sum(c.count for c in kids)

    # FOLLOW-UP 2: only the top k completions
    def suggest(self, prefix, k=3):
        # first k completions in alphabetical order; the
        # DFS
        # visits children in order and stops after k words
        node = self.walk(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack and len(out) < k:
            n, text = stack.pop()
            if n.is_word:
                out.append(text)
            # reversed so the smallest letter is popped
            # first
            for ch in sorted(n.children, reverse=True):
                stack.append((n.children[ch], text + ch))
        return out

    def completions(self, prefix):
        # every word with the prefix
        return self.suggest(prefix, k=10 ** 9)


if __name__ == "__main__":
    t = PrefixTrie(["apple", "app", "apricot", "banana",
                    "apple"])
    ap = t.completions("ap")
    assert ap == ["app", "apple", "apricot"]
    everything = t.completions("")
    assert everything[-1] == "banana"
    assert t.completions("x") == []
    assert t.count_prefix("ap") == 3
```

```
    assert t.count_prefix("b") == 1
    assert t.count_prefix("zz") == 0
    # the top 2 in alphabetical order
    assert t.suggest("ap", 2) == ["app", "apple"]
    assert t.count_prefix("") == 4
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], queries: list[str]) -> list
[list[str]]:
SAMPLE INPUT
{"words":["car","cat","dog"],"queries":["ca","d"]}
```

Test cases

Input	Expected
{"words":["car","cat","dog"],"queries":["ca","d"]}	[["car","cat"],["dog"]]

Input	Expected
`{"words":[],"queries":["a"]}`	`[[]]`

ALSO REPORTED · 1 more

Trie Prefix Search / Autocomplete | 1Point3Acres | tech-screen, onsite-coding | asked 2026-07-02 | Medium

https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

Preparation: Write the trie class from memory in under 12 min. Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY PATH: question 64 of 167 (Core 90)

Priority score 3.82 | Data structure and object design | asked 67 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 2 of 6

Implement a Reusable Locker Allocation System

POSTED 2026-08-14 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 8.1** | **30 DAYS 0** | **PRIORITY 167/228**

ABOUT THIS QUESTION

Topic: Core 90 - Data structure and object design

Optimal answer from: the interview site's own solution

LeetCode match: LC 1845 Seat Reservation Manager

Posted on PracHub: 2026-08-14 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

QUESTION

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'],
    ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

APPROACH

Min-heap of released locker numbers plus a counter for the next never-used number; store pops the smallest, pickup pushes back. Track the last successful store separately. O(log n) per op.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what store and pickup return, what happens when a customer stores twice, when no locker is free, and whether numbers are reused smallest first.

BRUTE FORCE TO SAY OUT LOUD

Scan lockers 1..n for the first free one on every store: O(n) per operation.

OPTIMAL IDEA

Keep a min-heap of released locker numbers and a counter next_new for lockers never used. store: if the customer already has one return -1; take the heap minimum if the heap is not empty, otherwise next_new if it is at most n, else return -1; remember customer to locker in a dict. pickup: free the customer's locker by pushing it on the heap. O(log n) per operation, and memory depends on used lockers, not n.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Pickup then store reuses the smallest freed number. Double store by one customer. Store when full. Pickup with no locker.

LIKELY FOLLOW-UPS

n up to a billion (still fine because of the counter), thread safety with a lock, or lockers with sizes.

PITFALLS

Initializing the heap with all n numbers. Not removing the customer record on pickup.

HOW TO THINK ABOUT IT

'Smallest free number' with numbers coming back means a min-heap of returned numbers. Numbers that were never handed out do not need to be stored: keep a counter for the next unused number and use the heap first. That keeps memory proportional to the number of lockers in use, not to n.

Basis: Written; the smallest-available-number pattern (LeetCode 1845 Seat Reservation Manager, 2336 Smallest Number in Infinite Set).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
owner = {}  # customer -> locker
def store(c):
    if c in owner:
        return -1
    taken = set(owner.values())
    for locker in range(1, n + 1):
        if locker not in taken:
            owner[c] = locker
            return locker
    return -1
def pickup(c):
    owner.pop(c, None)
```

TIME	SPACE
O(n) per operation	O(n)

WHY IT IS SLOW

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION · Min-heap of released numbers plus a next-unused counter

TIME	SPACE
O(log n) per operation	O(lockers released and not reused)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. **Cost:** O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

FOLLOW-UP 2 Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

FOLLOW-UP 3 Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement a Reusable Locker Allocation
# System
#
# ## Problem A locker bank has lockers numbered from 1
# through n. Process operations in order. store(customer)
# assigns the smallest available locker and returns its
# number, or -1 if that customer already has a locker or
# none is available. pickup(customer) releases and returns
# that customer's locker, or -1 if no locker is assigned.
# last() returns the locker number from the most recent
# successful store, or -1 if there has not been one.
# Return one result per operation. ### Constraints &
# Assumptions - 1 <= n <= 200,000. - There are at most
# 200,000 operations. - Customer IDs are non-empty
# strings. - A released locker becomes immediately
# available for reuse. ### Clarifications - Failed store
# and pickup operations do not change last(). - The
# allocation rule is always the smallest currently free
# locker number. - A customer can store again after a
# successful pickup.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are 10^9 lockers.
# 2. Many threads call store and pickup.
# 3. Lockers of different sizes.
#
# INTERVIEW NOTES
# 'Smallest free number' with numbers coming back means a
# min-heap of returned numbers. Numbers that were never
# handed out do not need to be stored: keep a counter for
# the next unused number and use the heap first. That
# keeps memory proportional to the number of lockers in
# use, not to n.
#
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one
# Keep an array of owners and scan from 1 for the first
# empty slot on store. Obviously correct, and each store
# can touch every locker. Time O(n) per operation. Space
```

```
# O(n).


import heapq
```

```
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one | time O(n) per operation
class LockersBrute:
    def __init__(self, n):
        self.owner, self.of = [None] * (n + 1), {}

    def store(self, customer):
        if customer in self.of:
            return -1
        for i in range(1, len(self.owner)):
            if self.owner[i] is None:
                self.owner[i], self.of[customer] = (
                    customer,
                    i,
                )
                return i
        return -1

    def pickup(self, customer):
        i = self.of.pop(customer, None)
        if i is None:
            return -1
        self.owner[i] = None
        return i


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
# else -1.
class Lockers:
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
```

```
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
```

```
        return locker

    def last(self):
        # failed operations never change this
        return self.last_store


# FOLLOW-UP 2: Many threads call store and pickup.
# Answer: Protect the heap, the counter and the customer
# map with one lock so a locker number is never handed out
# twice (or fine-grained: a lock around the (heap,
# counter) pair).
class SafeLockers(Lockers):
    """Many threads: one lock around every operation."""

    def __init__(self, n):
        import threading
        super().__init__(n)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)
```

```
    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)


# HELPERS for the checks below
def process(n, operations):
    box, out = Lockers(n), []
    for op in operations:
        if op[0] == "store":
            out.append(box.store(op[1]))
        elif op[0] == "pickup":
            out.append(box.pickup(op[1]))
        else:
            out.append(box.last())
    return out


if __name__ == "__main__":
    ops = [["store", "a"], ["store", "b"],
           ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, ops) == [1, 2, 1, 1, 1]
    # nothing stored yet, and a missing pickup
    empty = [["last"], ["pickup", "x"]]
    assert process(2, empty) == [-1, -1]
    # full, and a customer who already has a locker
    full = [["store", "a"], ["store", "a"],
            ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]
    # failed operations leave last() alone
    more = full + [["last"]]
    assert process(2, more)[-1] == 2
    # a huge locker bank costs nothing up front
    one = [["store", "a"], ["last"]]
    assert process(10 ** 9, one) == [1, 1]
    s = SafeLockers(2)
    assert s.store("a") == 1 and s.pickup("a") == 1
```

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[lis
t[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
`(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])`	`[1, 2, 1, 1, 1]`
`(5, [])`	`[]`

ALSO REPORTED · 1 more

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

STUDY PATH: question 65 of 167 (Core 90)

Priority score 3.79 | Data structure and object design | asked 233 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 3 of 6

★ TOP 75: rank 73 in the Medium and Hard priority list

★ Design a restaurant waitlist system

POSTED 2026-03-01 | 233 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.3 | 30 DAYS 0 | PRIORITY 49/228

ABOUT THIS QUESTION

Topic: Core 90 - Data structure and object design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #146 LRU Cache

LeetCode match: No direct match; related LC 146 LRU Cache (ordered map)

Posted on PracHub: 2026-03-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/design-a-restaurant-waitlist-system

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **`c`** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**
   - Adds a new party to the end of the waitlist and returns its id.
2. **`cancelParty(partyId) -> bool`**
   - Removes the party from the waitlist if present.
   - Returns **`true`** if removed, otherwise **`false`**.
3. **`seatTable(capacity) -> partyId or null`**
   - Finds and removes the earliest party with **`size <= capacity`**.
   - Returns that party's id, or **`null`** if nobody fits.
4. **`getPosition(partyId) -> int`**
   - Returns how many parties are ahead of this party in the current waitlist order (0-based).
   - If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.

QUESTION

- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

APPROACH

Hash map partyId -> party plus an ordered queue (linked list/OrderedDict) for arrival order; on a free table of size s, seat the earliest waiting party that fits; cancel is O(1) with lazy deletion.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact API (add party, cancel, seat when a table of capacity c frees up), what happens when no party fits, and whether a larger table may seat a smaller party (yes, if size <= c).

BRUTE FORCE TO SAY OUT LOUD

Keep a list in arrival order; on a table of capacity c scan from the front for the first party that fits. O(n) per seating, O(n) per cancel.

OPTIMAL IDEA

Keep id to party in a dict. For efficiency use a segment tree over arrival order storing the minimum party size in each range; the earliest party that fits is the leftmost position with size <= c, found by descending the tree in O(log n). Cancel and seat set that position's size to infinity. The simpler version uses an ordered dict plus lazy deletion and scans, O(n) in the worst case, which is fine to start with and to improve.

DRY-RUN THESE CASES

A table that fits nobody, a party that cancels before being seated, ties on arrival, a table that fits the second party but not the first, duplicate party id.

LIKELY FOLLOW-UPS

Large parties waiting too long (starvation), time-based expiry, or estimating wait time.

PITFALLS

Skipping a party that cancelled but is still in the queue. Choosing the best fit instead of the earliest fit.

HOW TO THINK ABOUT IT

'Earliest arrival whose size fits' is a query on an ordered sequence: find the leftmost position whose value is at most c. A scan does it in O(n); a segment tree that stores the minimum size over each range answers it in O(log n) by walking down to the leftmost child whose range minimum fits. Cancelling or seating simply sets that position to infinity.

HOW TO THINK ABOUT IT

Basis: Written design (compare LeetCode 146 for the ordered-map idea). API assumed (confirm it): add(id, name, size), cancel(id), seat(capacity) seats and returns the earliest-arrived waiting party whose size is at most the capacity, or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parties = []  # in arrival order
def join(party):
    parties.append(party)
def seat(capacity):
    for party in parties:
        if party.size <= capacity:
            parties.remove(party)
            return party
    return None
def cancel(party_id):
    for party in parties:
        if party.id == party_id:
            parties.remove(party)
            return
```

TIME	SPACE
O(n) per seat and per cancel	**O(n)**

WHY IT IS SLOW

Both operations walk the list, O(n). A segment tree of the smallest size per range finds the leftmost party that fits in O(log n).

OPTIMAL SOLUTION · Segment tree of minimum size over arrival order

TIME	SPACE
O(log n) per operation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Parties that wait too long should be seated first (starvation).

Add an aging rule: if the earliest party has waited longer than a threshold and a table of at least its size appears, seat it before anyone else. The segment tree can query 'leftmost size <= c' with c = the table, so order is preserved.

FOLLOW-UP 2 Seat the best-fitting party, not the earliest.

That is a different policy: keep sizes in a sorted structure of (size, arrival) and take the largest size at most the capacity (ties by arrival). A sorted container with bisect gives O(log n) lookups.

FOLLOW-UP 3 Estimate the waiting time of a party.

Track the arrival rate and typical table turnover per capacity; the estimate is the expected time until enough tables of at least that size free up, ahead of the party in the queue.

FOLLOW-UP 4 Seat the best-fitting party (the largest that fits), not the earliest.

Group waiting parties by size in queues (arrival order inside a size) and keep the sizes in a sorted list: the best fit for a table of capacity c is the largest size at most c, found with bisect, then the oldest party of that size. O(log distinct sizes) per seating. Verified against a linear scan.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design a restaurant waitlist system
#
# You are implementing the waitlist system for a
# restaurant. Parties arrive over time, can cancel, and
# get seated when a table becomes available. ## Rules -
# Each party has a unique partyId, a name, and a size
# (number of people). - Parties are seated in **arrival
# order**, but only if they **fit** the available table. -
# When a table of capacity c becomes available, you must
# seat the **earliest-arrived** party whose size <= c. -
# If no party fits, the table remains unused for that
# event. ## Operations to support Design a data structure
# / class that supports the following operations
# efficiently: 1. addParty(name, size) -> partyId - Adds a
# new party to the end of the waitlist and returns its id.
# 2. cancelParty(partyId) -> bool - Removes the party from
# the waitlist if present. - Returns true if removed,
# otherwise false. 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Parties that wait too long should be seated first
# (starvation).
# 2. Seat the best-fitting party, not the earliest.
# 3. Estimate the waiting time of a party.
# 4. Seat the best-fitting party (the largest that fits),
# not the earliest.
#
# INTERVIEW NOTES
# 'Earliest arrival whose size fits' is a query on an
# ordered sequence: find the leftmost position whose value
# is at most c. A scan does it in O(n); a segment tree
# that stores the minimum size over each range answers it
# in O(log n) by walking down to the leftmost child whose
# range minimum fits. Cancelling or seating simply sets
# that position to infinity.
#
# BRUTE FORCE (say it first): Scan the waiting list in
# arrival order
# Keep parties in arrival order in a list. seat scans from
# the front for the first party whose size fits; cancel
# removes by id. Correct and the oracle; every operation
# walks the list. Time O(n) per seat and per cancel. Space
# O(n).
```

```python
from bisect import bisect_right, insort
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Scan the waiting list in
# arrival order | time O(n) per seat and per cancel
class WaitlistBrute:
    def __init__(self):
        self.parties = []

    def add(self, pid, name, size):
        self.parties.append((pid, name, size))

    def cancel(self, pid):
        before = len(self.parties)
        self.parties = [
            p for p in self.parties if p[0] != pid
        ]
        return len(self.parties) < before

    def seat(self, capacity):
        for i, p in enumerate(self.parties):
            if p[2] <= capacity:
                del self.parties[i]
                return p[0]
        return None


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Segment tree of minimum size
# over arrival order
# Idea: Each arrival gets the next position. The tree
# stores, for each range, the smallest waiting party size
# (infinity for empty or seated slots).
class Waitlist:
    """Earliest party that fits: a segment tree of the
    smallest size over arrival slots."""

    def __init__(self, capacity=1024):
        self.cap = capacity
        self.tree = [INF] * (2 * capacity)
        self.count = 0
```

```python
        self.slot, self.who = {}, {}

    def update(self, i, value):
        i += self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = min(self.tree[2 * i],
                               self.tree[2 * i + 1])

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        i = self.slot.get(party_id)
        if i is None or self.tree[i + self.cap] == INF:
            return False
        self.update(i, INF)
        return True

    def seat(self, capacity):
        # no party small enough: nobody is seated
        if self.tree[1] > capacity:
            return None
        i = 1
```

```python
        # go left whenever the left half holds a party
        # that fits
        while i < self.cap:
            left = self.tree[2 * i] <= capacity
            i = 2 * i if left else 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]


# FOLLOW-UP 2: Seat the best-fitting party, not the
# earliest.
# Answer: That is a different policy: keep sizes in a
# sorted structure of (size, arrival) and take the largest
# size at most the capacity (ties by arrival). A sorted
# container with bisect gives O(log n) lookups.
# FOLLOW-UP 4: Seat the best-fitting party (the largest
# that fits), not the earliest.
```

```python
# Answer: Group waiting parties by size in queues (arrival
# order inside a size) and keep the sizes in a sorted
# list: the best fit for a table of capacity c is the
# largest size at most c, found with bisect, then the
# oldest party of that size.
class BestFit:
    """Policy B: the largest party that fits the table."""

    def __init__(self):
        # size -> parties of that size, oldest first
        self.queues = defaultdict(deque)
        self.sizes = []

    def add(self, party_id, size):
        if not self.queues[size]:
            insort(self.sizes, size)
        self.queues[size].append(party_id)

    def seat(self, capacity):
        k = bisect_right(self.sizes, capacity)
        if k == 0:
            return None
        size = self.sizes[k - 1]
        party = self.queues[size].popleft()
        if not self.queues[size]:
            self.sizes.pop(k - 1)
        return party


if __name__ == "__main__":
    w = Waitlist(8)
    w.add("p1", "Ann", 4)
    w.add("p2", "Bob", 2)
    w.add("p3", "Cy", 6)
    # a table for 3: the earliest party that fits is Bob
    assert w.seat(3) == "p2"
    # cancel Ann; a table for 6 now seats Cy
    assert w.cancel("p1") and not w.cancel("p1")
    assert w.seat(6) == "p3"
    assert w.seat(10) is None
    b = BestFit()
    parties = (("a", 2), ("b", 4), ("c", 4), ("d", 6))
    for pid, size in parties:
        b.add(pid, size)
    # a table for 5: the biggest fit is size 4, oldest
```

```python
    # first
    assert b.seat(5) == "b"
    assert b.seat(5) == "c"
    assert b.seat(5) == "a"
    assert b.seat(1) is None
```

FUNCTION AND TEST CASES

Test cases

Input	Expected
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('seatTable', 2), ('seatTable', 4)]`	`[1, 2, 2, 1]`
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('addParty', 'C', 3), ('getPosition', 3), ('seatTable', 2), ('getPosition', 3), ('cancelParty', 1), ('getPosition', 3), ('cancelParty', 1), ('seatTable', 2), ('seatTable', 3), ('getPosition', 3)]`	`[1, 2, 3, 2, 2, 1, True, 0, False, None, 3, -1]`

ALSO REPORTED · 1 more

Restaurant Waitlist: join / delete / find_first_match | 1Point3Acres | onsite-coding | asked 2026-02-01 | Single report

https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

Preparation: Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY PATH: question 66 of 167 (Core 90)

Priority score 3.05 | Data structure and object design | asked 287 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 4 of 6

★ TOP 75: rank 36 in the Medium and Hard priority list

★ Design a logger that returns top-K messages

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 33** | **PRIORITY 33/228**

ABOUT THIS QUESTION

Topic: Core 90 - Data structure and object design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter, LeetCode #692 Top K Frequent Words

LeetCode match: LC 359 Logger Rate Limiter + LC 692 Top K Frequent Words (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/design-a-logger-that-returns-top-k-messages

QUESTION

Overview: This question evaluates skills in designing in-memory data structures and algorithms for frequency counting and interval querying, including extracting top-K elements with a heap, within the Coding & Algorithms domain.

APPROACH

Hash map message -> count (plus a deque of timestamps for interval queries); top-K with a size-k heap, ties broken lexicographically.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask what the API is: log(timestamp, message), top-K over all time or over an interval, whether duplicates within a time gap are suppressed (the LeetCode 359 rate limiter), and how ties break (lexicographically, for LeetCode 692).

BRUTE FORCE TO SAY OUT LOUD

Store every entry in a list and sort or scan on each query. O(n log n) per query.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

A dict message to count (and, for interval queries, a dict message to a list of timestamps kept in order so a time window is two binary searches). Top-K uses a heap of size K with the key (count descending, message ascending): heapq.nsmallest(k, items, key=(-count, message)). The rate limiter part is a dict message to last printed time, allowing a message when the gap is at least 10 seconds. O(d log K) per top-K query.

DRY-RUN THESE CASES

Fewer than K distinct messages, ties on count, the same message repeated inside the rate-limit window, an empty logger.

LIKELY FOLLOW-UPS

Streaming top-K with bounded memory (Space-Saving), or evicting old entries.

PITFALLS

Not specifying the tie rule. Using the full sort when only K items are needed.

HOW TO THINK ABOUT IT

Store enough to answer both questions without scanning everything. A dict from message to its list of timestamps (appended in time order) answers 'how many in [start, end]' with two binary searches, and a heap of size K over (count, message) gives the top K. Say how the logger handles repeated messages and what 'top' means with ties.

Basis: Written (compare LeetCode 359 Logger Rate Limiter and 692 Top K Frequent Words). Assumed API (confirm it): log(timestamp, message); top_k(k, start, end) over an optional time window; ties by message alphabetically.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
logs = []  # (timestamp, message)
def top_k(k, start, end):
    counts = Counter()
    for ts, msg in logs:
        if start <= ts <= end:
            counts[msg] += 1
    def rank(m):
        return (-counts[m], m)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d) per query	**O(n)**

WHY IT IS SLOW

Every query reads the whole log, O(n + d log d). Per-message sorted timestamps with binary search answer a window count in O(log n), and a heap picks the top k.

OPTIMAL SOLUTION · Per-message sorted timestamps plus a heap for the top K

TIME	SPACE
O(1) per log, O(d log n + d log k) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print a message only if it was not printed in the last 10 seconds (LeetCode 359).

Keep a dict message to the time it was last printed and compare: print if the gap is at least 10 (and update the time only when you print). O(1) per call.

FOLLOW-UP 2 Memory must stay bounded.

Expire timestamps older than the largest window you support (drop from the front of each list, delete messages with no timestamps left), or approximate counts with a count-min sketch plus a heap for the heavy hitters. **Cost:** O(1) amortized to drop expired timestamps; memory O(messages inside the window).

FOLLOW-UP 3 Very high top_k frequency of calls.

Maintain counts per message incrementally and a top-K structure (a heap with lazy updates or a sorted structure) so top_k costs O(k) instead of O(d log k); windowed queries still need per-message timestamps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design a logger that returns top-K
# messages
#
# This question evaluates skills in designing in-memory
# data structures and algorithms for frequency counting
# and interval querying, including extracting top-K
# elements with a heap, within the Coding & Algorithms
# domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print a message only if it was not printed in the
# last 10 seconds (LeetCode 359).
# 2. Memory must stay bounded.
# 3. Very high top_k frequency of calls.
#
# INTERVIEW NOTES
# Store enough to answer both questions without scanning
# everything. A dict from message to its list of
# timestamps (appended in time order) answers 'how many in
# [start, end]' with two binary searches, and a heap of
# size K over (count, message) gives the top K. Say how
# the logger handles repeated messages and what 'top'
# means with ties.
#
# BRUTE FORCE (say it first): Keep every log entry and
# scan it for each query
# Store all (timestamp, message) pairs; for a query count
# the messages inside the window and sort. Correct and the
# oracle; every query reads the whole log. Time O(n + d
# log d) per query. Space O(n).


from collections import Counter
import heapq
from bisect import bisect_left, bisect_right
from collections import defaultdict


# BRUTE FORCE (say it first): Keep every log entry and
# scan it for each query | time O(n + d log d) per query
class LoggerBrute:
    def __init__(self):
        self.entries = []

    def log(self, ts, msg):
        self.entries.append((ts, msg))

    def top_k(self, k, start=None, end=None):
        c = Counter(
            m
            for t, m in self.entries
            if (start is None or t >= start)
            and (end is None or t <= end)
        )
        return sorted(c, key=lambda m: (-c[m], m))[:k]


# BASE QUESTION (optimal): Per-message sorted timestamps
# plus a heap for the top K
# Idea: log appends the timestamp to the message's list
# (timestamps arrive in order, so lists stay sorted). A
# window count for one message is bisect_right(end) -
# bisect_left(start).
class Logger:
    def __init__(self):
        # message -> its timestamps, in order of arrival
        self.times = defaultdict(list)

    def log(self, ts, msg):
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        # the k most frequent messages in [start, end]
        counts = {}
        for msg, ts in self.times.items():
            lo, hi = 0, len(ts)
            if start is not None:
                lo = bisect_left(ts, start)
            if end is not None:
                hi = bisect_right(ts, end)
            if hi > lo:
                counts[msg] = hi - lo
        # ties are broken alphabetically
        def rank(m):
            return (-counts[m], m)
        return heapq.nsmallest(k, counts, key=rank)

    def expire(self, before):
        # bounded memory: forget timestamps older than
        # `before`
        for msg in list(self.times):
            ts = self.times[msg]
            self.times[msg] = ts[bisect_left(ts, before):]
```

```
            if not self.times[msg]:
                del self.times[msg]


# FOLLOW-UP 1: Print a message only if it was not printed
# in the last 10 seconds (LeetCode 359).
# Answer: Keep a dict message to the time it was last
# printed and compare: print if the gap is at least 10
# (and update the time only when you print). O(1) per
# call.
class RateLimiter:
    """LeetCode 359: one print per message per gap."""

    def __init__(self, gap=10):
        self.gap, self.last = gap, {}

    def should_print(self, ts, msg):
        last = self.last.get(msg)
        if last is not None and ts - last < self.gap:
            return False
        # only a printed message starts a new window
        self.last[msg] = ts
        return True


if __name__ == "__main__":
    log = Logger()
    events = [(1, "a"), (2, "b"), (3, "a"),
              (4, "c"), (5, "a"), (6, "b")]
    for ts, msg in events:
        log.log(ts, msg)
    assert log.top_k(2) == ["a", "b"]
    # only the window [4, 6]: a once, b once, c once
    assert log.top_k(3, 4, 6) == ["a", "b", "c"]
    assert log.top_k(2, 100, 200) == []
    log.expire(4)
    assert log.top_k(5) == ["a", "b", "c"]
    r = RateLimiter()
    times = (1, 2, 11, 12, 21)
    got = [r.should_print(t, "x") for t in times]
    assert got == [True, False, True, False, True]
```

FUNCTION AND TEST CASES

Signature

```
def solution(logs: list[tuple[int, str]], queries: list[li
st[int]]) -> list[list[str]]:
```

Sample call

```
solution([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3
, 2), (1, 10, 3)])
```

Test cases

Input	Expected
`([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3, 2), (1, 10, 3)])`	`[['A', 'B'], ['A', 'B', 'C']]`
`([(1, 'dog'), (2, 'cat'), (2, 'dog'), (4, 'cat'), (5, 'ant')], [(1, 4, 2)])`	`[['cat', 'dog']]`

STUDY PATH: question 67 of 167 (Core 90)

Priority score 2.86 | Data structure and object design | asked 292 days ago

MEDIUM | IN APP LC 380 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 5 of 6

Design set with O(1) random access

POSTED 2026-01-01 | 292 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 74/228

ABOUT THIS QUESTION

Topic: Core 90 - Data structure and object design

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1)

Posted on PracHub: 2026-01-01 (updated 2026-06-01)

Original:
https://prachub.com/coding-questions/design-set-with-o-1-random-access

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.

QUESTION

- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints
1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom'
    ,'add','getRandom'], [[1],[2],[],[1],[],[2],[]],
    7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','g
    etRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True,
    None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

APPROACH

Array of values + hash map value -> index; remove by swapping with the last element and popping.

HOW TO THINK ABOUT IT

A hash set gives O(1) add and delete but cannot pick a random element in O(1); an array can pick a random element but deleting from the middle is O(n). Combine them: keep the values in an array and a dict from value to array index. To delete, swap the element with the last one, pop the end, and update the moved element's index.

Basis: LeetCode 380, Insert Delete GetRandom O(1) (FancySet). Choice made: getRandom on an empty set raises an error (a defined, loud failure).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def add(x):
    if x in items:
        return False
    items.append(x)
    return True
def delete(x):
    if x not in items:
        return False
    items.remove(x)
    return True
def get_random():
    return random.choice(items)
```

TIME	SPACE
O(n) for add and delete, O(1) getRandom	**O(n)**

WHY IT IS SLOW

Membership checks and deletion are O(n). An array plus a dictionary from value to index, deleting by swapping with the last element, makes all three operations O(1).

OPTIMAL SOLUTION · Array of values plus a dict from value to index

TIME	SPACE
O(1) average per operation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make it thread-safe: one lock or finer-grained locking?

One lock around every operation is simplest and correct (delete touches the array, the dict and a second element's index together, so they must change as one unit). Finer locks do not help much because every operation touches shared structure; use sharding by value hash if contention is the problem.

FOLLOW-UP 2 What should getRandom do on an empty set?

Raise a clear error (as here) or return a sentinel such as None; whichever you pick, document it and make the caller handle it. Returning an arbitrary value hides bugs.

FOLLOW-UP 3 Duplicates allowed (a multiset, LeetCode 381).

Map each value to a set of indices in the array; delete swaps with the last element and fixes both index sets. getRandom stays a uniform pick over array positions, so values are chosen in proportion to their counts.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design set with O(1) random access
#
# Design a data structure ("FancySet") that stores unique
# integers and supports the following operations, each in
# average O(1) time: - add(x): Insert value x if it is not
# already present. Return whether the insert happened. -
# delete(x): Remove value x if it exists. Return whether a
# deletion happened. - getRandom(): Return a uniformly
# random element currently in the set. Requirements /
# follow-ups: - All operations should be average O(1). -
# Handle corner cases such as calling getRandom() on an
# empty set (define and justify behavior). - Discuss how
# you would make this data structure thread-safe. Compare
# using a single lock for the whole set vs finer-grained
# locking (e.g., element-level locks), and explain trade-
# offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make it thread-safe: one lock or finer-grained
# locking?
# 2. What should getRandom do on an empty set?
# 3. Duplicates allowed (a multiset, LeetCode 381).
#
# INTERVIEW NOTES
# A hash set gives O(1) add and delete but cannot pick a
# random element in O(1); an array can pick a random
# element but deleting from the middle is O(n). Combine
# them: keep the values in an array and a dict from value
# to array index. To delete, swap the element with the
# last one, pop the end, and update the moved element's
# index.
#
# BRUTE FORCE (say it first): A plain list
# Store values in a list; add checks for membership with a
# scan and delete removes by search. Correct and the
# oracle; membership and deletion are linear. Time O(n)
# for add and delete, O(1) getRandom. Space O(n).


import random
import threading


# BRUTE FORCE (say it first): A plain list | time O(n) for
# add and delete, O(1) getRandom
```

```
class FancySetBrute:
    def __init__(self):
        self.a = []
```

```
    def add(self, x):
        if x in self.a:
            return False
        self.a.append(x)
        return True

    def delete(self, x):
        if x not in self.a:
            return False
        self.a.remove(x)
        return True

    def getRandom(self):
        if not self.a:
            raise IndexError("empty set")
        return random.choice(self.a)


# BASE QUESTION (optimal): Array of values plus a dict
# from value to index
# Idea: add appends to the array and records the index.
# delete finds the index from the dict, moves the last
# element into that slot, updates its index in the dict,
# pops the array and removes the key.
class FancySet:
    """add, delete and getRandom, each O(1) on average."""

    def __init__(self, rng=None):
        # the values in a list (for random access) and a
        # map
        # from value to its index in that list
        self.items, self.pos = [], {}
        self.rng = rng or random

    def add(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
        self.items.append(x)
        return True
```

```
    def delete(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.items[-1]
        # move the last item into the hole, then drop the
        # tail
        self.items[i] = last
        self.pos[last] = i
        self.items.pop()
        del self.pos[x]
        return True

    def get_random(self):
        if not self.items:
            # say what happens on an empty set
            raise IndexError("empty set")
        i = self.rng.randrange(len(self.items))
        return self.items[i]


# FOLLOW-UP 1: Make it thread-safe: one lock or finer-
# grained locking?
# Answer: One lock around every operation is simplest and
# correct (delete touches the array, the dict and a second
# element's index together, so they must change as one
# unit).
class SafeSet(FancySet):
    """One lock: delete changes three things at once."""

    def __init__(self, rng=None):
        super().__init__(rng)
```

```
        self.lock = threading.Lock()

    def add(self, x):
        with self.lock:
            return super().add(x)

    def delete(self, x):
        with self.lock:
            return super().delete(x)

    def get_random(self):
        with self.lock:
            return super().get_random()
```

```
# FOLLOW-UP 3: Duplicates allowed (a multiset, LeetCode
# 381).
# Answer: Map each value to a set of indices in the array;
# delete swaps with the last element and fixes both index
# sets. getRandom stays a uniform pick over array
# positions, so values are chosen in proportion to their
# counts.
class RandomizedCollection:
    """Duplicates allowed (LeetCode 381)."""

    def __init__(self):
        self.items = []
        # value -> set of indices where it sits in the
        # list
        self.where = {}

    def insert(self, x):
        first = x not in self.where or not self.where[x]
        spots = self.where.setdefault(x, set())
        spots.add(len(self.items))
        self.items.append(x)
        return first

    def remove(self, x):
        if not self.where.get(x):
            return False
        i = self.where[x].pop()
        end = len(self.items) - 1
        if i != end:
            # move the last item into the hole and fix its
            # index set
            last = self.items[end]
            self.items[i] = last
            self.where[last].discard(end)
            self.where[last].add(i)
        self.items.pop()
        return True

    def get_random(self):
        return random.choice(self.items)


if __name__ == "__main__":
    s = FancySet(random.Random(1))
```

```
    assert s.add(1) and s.add(2) and not s.add(1)
    assert s.delete(1) and not s.delete(1)
    assert s.get_random() == 2
    s.delete(2)
    try:
        s.get_random()
        raise AssertionError("empty set should raise")
    except IndexError:
        pass
    # every value is picked about equally often
    big = FancySet(random.Random(5))
    for v in range(4):
        big.add(v)
```

```
    picks = [big.get_random() for _ in range(4000)]
    for v in range(4):
        assert 850 < picks.count(v) < 1150
    # deleting from the middle keeps the map consistent
    for v in (0, 2):
        big.delete(v)
    assert sorted(big.items) == [1, 3]
    for v, i in big.pos.items():
        assert big.items[i] == v
    safe = SafeSet()
    assert safe.add(7) and safe.get_random() == 7
    rc = RandomizedCollection()
    assert rc.insert(5)
    assert not rc.insert(5)
    assert rc.insert(6)
    assert rc.remove(5) and rc.remove(5)
    assert not rc.remove(5)
    assert rc.items == [6]
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[str], values: list[list[int]], seed: int) -> str:
```

Sample call

```
solution(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
```

Test cases

Input	Expected
`(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)`	`[True, True, 1, True, 2, False, 2]`
`(['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)`	`[None, True, False, False, True, None]`

STUDY PATH: question 68 of 167 (Core 90)

Priority score 2.5 | Data structure and object design | asked 240 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 6 of 6

Determine Reporting Relationships

POSTED 2026-02-22 | 240 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.4 | 30 DAYS 33 | PRIORITY 63/228

ABOUT THIS QUESTION

Topic: Core 90 - Data structure and object design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: No direct match; related LC 721 Accounts Merge (union-find)

Posted on PracHub: 2026-02-22 (updated 2026-04-21)

Original: https://prachub.com/coding-questions/determine-reporting-relationships

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.

QUESTION

- An employee is not considered their own manager.

Constraints

0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3),
    ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4),
    ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

APPROACH

Manager edges as a directed graph/parent map; peers via union-find or shared-manager sets; queries answer reachability up the manager chain. Keep both structures in sync.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether is_manager means direct manager or any manager up the chain (the example implies the chain), whether peers can be added before their manager is known, and whether input is always consistent (the statement says yes).

BRUTE FORCE TO SAY OUT LOUD

Store all relations and recompute reachability with a graph search on each query. O(V + E) per query.

OPTIMAL IDEA

Keep a Union-Find over employees for peer groups and attach one manager to each group. add_manager(a, b) sets the manager of b's group to a. add_peer(a, b) unions the two groups and, if both had managers, keeps the (necessarily identical) manager. is_manager(a, b): look up the manager of b's group, return true if it is a, otherwise continue from that manager's group up the chain. Each query walks at most the depth of the hierarchy.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

add_manager(m, x) then add_peer(x, y) gives is_manager(m, y) true; add_peer before add_manager; a manager's manager (indirect); an employee with no manager; a query on unknown ids.

LIKELY FOLLOW-UPS

Direct manager only, the closest common manager, or removing relationships.

PITFALLS

Forgetting to merge managers when two groups unite. Treating peers as managers of each other.

HOW TO THINK ABOUT IT

Peers form groups (a peer relation is transitive: same manager as a person who has the same manager as another). A group has at most one direct manager, so store the manager on the group. To answer is_manager(a, b) go from b's group to its manager, then that manager's group to its manager, and so on, looking for a. Union-find makes the groups cheap to merge.

Basis: Written (compare LeetCode 721, Accounts Merge, for the union-find idea). Operations: add_manager(a, b) (a is b's direct manager), add_peer(a, b) (same direct manager), is_manager(a, b) (a is anywhere above b). Inferences must follow from earlier calls.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
managers, peers = [], []  # the stored facts
def is_manager(a, b):
    groups = merge_peers(peers)
    manager_of = {}
    for x, y in managers:  # x manages y
        boss = groups.find(x)
        manager_of[groups.find(y)] = boss
    node = groups.find(b)
    while node in manager_of:
        node = manager_of[node]
        if node == groups.find(a):
            return True
    return False
```

TIME	SPACE
O(F * employees) per query	**O(F)**

WHY IT IS SLOW

Each query redoes the whole inference over all facts. Union-Find for peer groups with a manager stored on each group keeps every update near-constant and a query O(depth).

OPTIMAL SOLUTION · Union-Find for peer groups with a manager stored on each group

TIME	SPACE
O(alpha) per update, O(depth) per query	**O(employees)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the closest common manager of two employees.

Collect the chain of managers above one employee into a set, then climb from the other until you meet it. O(depth).

FOLLOW-UP 2 Facts can be wrong or contradictory.

When uniting groups that already have different managers, the input is inconsistent: raise, or record a conflict instead of silently overwriting. Also guard the upward walk with a visited set so a cycle cannot loop forever.

FOLLOW-UP 3 Remove a relationship.

Union-Find cannot split groups; keep the facts and rebuild lazily, or store the peers as an explicit graph and recompute connected components on demand.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine Reporting Relationships
#
# Build an in-memory organization model that supports
# three operations on employee IDs: 1. add_manager(a, b):
# employee a is the direct manager of employee b. 2.
# add_peer(a, b): employees a and b share the same direct
# manager. 3. is_manager(a, b): return whether employee a
# is a manager of employee b. The answer should account
# for relationships that can be inferred from prior
# operations. For example, if add_manager(m, x) and
# add_peer(x, y) were previously recorded, then
# is_manager(m, y) should return true. You may assume the
# input is logically consistent and does not create
# invalid cycles in the org chart. Explain how you would
# design the data structures and implement these
# operations efficiently.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the closest common manager of two employees.
# 2. Facts can be wrong or contradictory.
# 3. Remove a relationship.
#
# INTERVIEW NOTES
# Peers form groups (a peer relation is transitive: same
# manager as a person who has the same manager as
# another). A group has at most one direct manager, so
# store the manager on the group. To answer is_manager(a,
# b) go from b's group to its manager, then that manager's
# group to its manager, and so on, looking for a. Union-
```

```
# find makes the groups cheap to merge.
#
# BRUTE FORCE (say it first): Recompute all inferences
# from the stored facts on every query
# Keep the list of facts. For a query, repeatedly merge
# peers into groups and attach managers until nothing
# changes, then walk up from b. Correct and the oracle;
# every query redoes the whole inference. Time O(F *
# employees) per query. Space O(F).


# BRUTE FORCE (say it first): Recompute all inferences
# from the stored facts on every query | time O(F *
# employees) per query
class OrgBrute:
    def __init__(self):
```

```
        self.mgr, self.peers = [], []

    def add_manager(self, a, b):
        self.mgr.append((a, b))

    def add_peer(self, a, b):
        self.peers.append((a, b))

    def is_manager(self, a, b):
        group = {}
        people = {
            x for p in self.mgr + self.peers for x in p
        }
        for x in people:
            group[x] = {x}
        for x, y in self.peers:
            if group[x] is not group[y]:
                merged = group[x] | group[y]
                for z in merged:
                    group[z] = merged
        boss = {}
        for m, x in self.mgr:
            for z in group[x]:
                boss[id(group[x])] = m
        cur, steps = b, 0
        while (
            cur in group
            and id(group[cur]) in boss
            and steps <= len(people)
        ):
            cur = boss[id(group[cur])]
            if cur == a:
                return True
            steps += 1
        return False


# BASE QUESTION (optimal): Union-Find for peer groups with
# a manager stored on each group
# Idea: Each employee starts in its own group. add_peer
# unions the two groups (and keeps the manager, if either
# has one). add_manager sets the manager of b's group to
# a.
# FOLLOW-UP 1: Return the closest common manager of two
# employees.
```

```
# Answer: Collect the chain of managers above one employee
# into a set, then climb from the other until you meet it.
# O(depth).
class Org:
    def __init__(self):
        # union-find over peers (people with the same
        # manager)
        self.parent = {}
        # peer-group root -> the manager of that group
        self.boss = {}
```

```
    def find(self, x):
        self.parent.setdefault(x, x)
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # two groups with different managers contradict
        if ra in self.boss and rb in self.boss \
                and self.boss[ra] != self.boss[rb]:
            # FOLLOW-UP 2: contradictory facts are
            # rejected
            raise ValueError("conflicting managers")
        self.parent[rb] = ra
        # the merged group keeps whichever manager is
        # known
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]

    def add_manager(self, a, b):
        # a is the direct manager of b (and b's peers)
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")
        self.boss[root] = a

    def manager_of(self, x):
        return self.boss.get(self.find(x))
```

```
    def is_manager(self, a, b):
        # climb b's chain of managers; a visited set stops
        # cycles
        seen, cur = set(), b
        while cur is not None and cur not in seen:
            seen.add(cur)
            cur = self.manager_of(cur)
            if cur == a:
                return True
        return False

    # FOLLOW-UP 1: the closest common manager
    def common_manager(self, a, b):
        # the closest manager above both people
        chain, cur = set(), self.manager_of(a)
        while cur is not None and cur not in chain:
            chain.add(cur)
            cur = self.manager_of(cur)
        cur, seen = self.manager_of(b), set()
        while cur is not None and cur not in seen:
            if cur in chain:
                return cur
            seen.add(cur)
            cur = self.manager_of(cur)
        return None


if __name__ == "__main__":
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    # qa shares dev's manager, so vp manages qa too
    assert o.is_manager("vp", "qa")
    assert o.is_manager("ceo", "qa")
    assert not o.is_manager("qa", "vp")
    assert not o.is_manager("dev", "qa")
    assert o.common_manager("dev", "qa") == "vp"
```

```
    assert o.common_manager("vp", "qa") == "ceo"
    # peers added before the manager is known
    p = Org()
    p.add_peer("a", "b")
    p.add_manager("boss", "a")
    assert p.is_manager("boss", "b")
    # contradictory facts are reported
```

```
    try:
        o.add_manager("ceo", "dev")
        raise AssertionError("should conflict")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[tuple[str, int, int]]) -> li
st[bool]:
SAMPLE INPUT
([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager"
, 1, 3)])
```

Test cases

Input	Expected
`([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])`	`[True]`
`([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4), ("is_manager", 2, 4), ("is_manager", 4, 2)])`	`[False, True, True, False]`